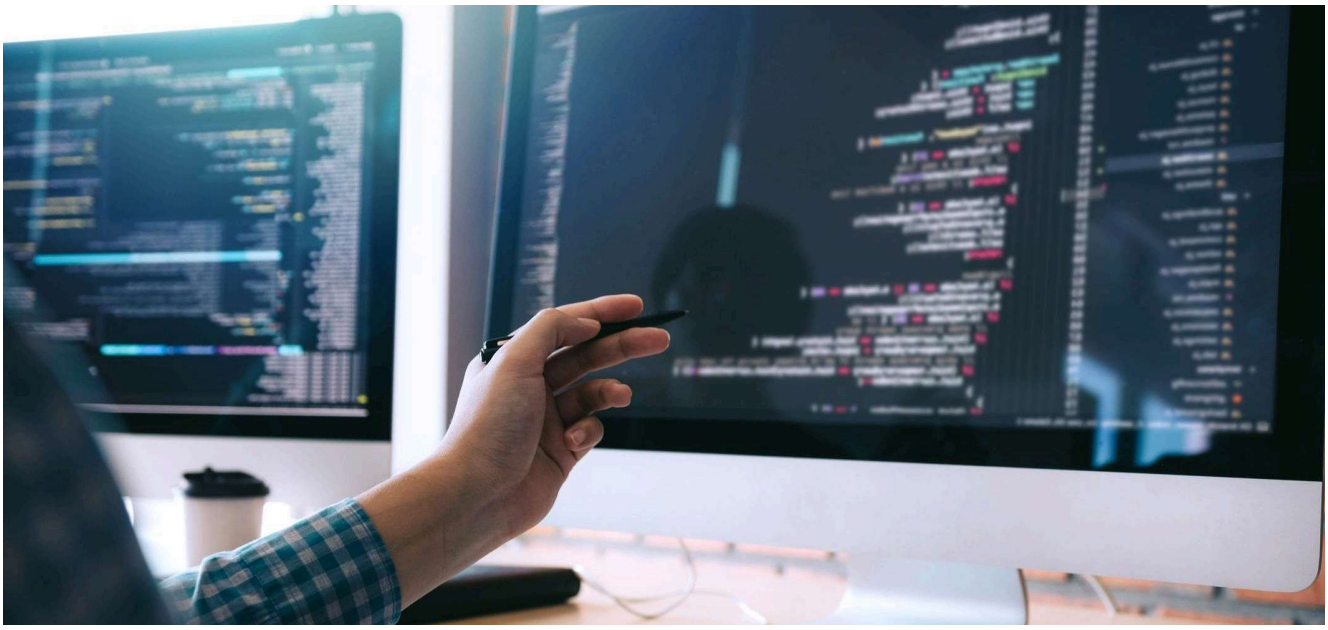
PROJECT: DEBUGGING A SALES DATA WORKFLOW



As a data engineer, you often face unexpected challenges in workflows. In this scenario, the `load_and_check()` function, in charge of managing sales data, encounters issues after the latest update. Unfortunately, your colleague who usually handles this code is currently on holiday, leaving you to troubleshoot.

Your task is to identify and address the issues in the sales data pipeline **without getting into every line of code.** The `load_and_check()` function loads the `sales.csv` dataset and performs several checks. Initially, it verifies the dataset's shape, ensuring it matches expectations. Subsequently, integrity checks are conducted to maintain data consistency and flag any anomalies.

The `sales.csv` dataset has various columns, focusing on critical fields such as `Total`, `Quantity`, `Unit price`, `Tax`, and `Date`. It's essential that the `Tax` column accurately represents 5% of the subtotal, calculated from the `Unit Price` multiplied by `Quantity`.

Your goal is to sort out the pipeline issues, aiming for the code to return 2 success messages upon completion. Make sure the input data remains accurate. Any necessary adjustments should be made inside the function. Do not manually edit the CSV file or pre-process the data outside of the function. Be mindful of updating any relevant if statements in the checks as needed.

```
import pandas as pd

def load_and_check():
    # Step 1: Load the data and check if it has the expected shape
    data = pd.read_csv('sales.csv')

    # Issue 1 fixed: Correct number of expected columns
    expected_columns = 17
    actual_columns = data.shape[1]
    if actual_columns != expected_columns:
        print(f"Data column mismatch! Expected {expected_columns}, but got
{actual_columns}.")
        print(f"Columns found: {list(data.columns)}")
    else:
        print("Data loaded successfully.")

    # Step 2: Calculate statistical values and merge with the original data
    grouped_data = data.groupby(['Date'])['Total'].agg(['mean', 'std'])
    grouped_data['threshold'] = 3 * grouped_data['std']
    grouped_data['max'] = grouped_data['mean'] + grouped_data.threshold
    grouped_data['min'] = grouped_data[['mean', 'threshold']].apply(lambda row:
max(0, row['mean'] - row['threshold']), axis=1)
    data = pd.merge(data, grouped_data, on='Date', how='left')

    # Issue 2 fixed:  Recalculating the 'Tax' column
    data['Tax'] = (data['Quantity'] * data['Unit price']).astype(float) * 0.05  #
Assuming tax is 5% of the subtotal

    # Condition_1 checks if 'Total' is within the acceptable range (min to max) for
each date
    data['Condition_1'] = (data['Total'] >= data['min']) & (data['Total'] <=
data['max'])
    data['Condition_1'].fillna(False, inplace=True)

    # Condition_2 checks if the 'Tax' column is properly calculated as 5% of
(Quantity * Unit price)
    data['Condition_2'] = round(data['Quantity'] * data['Unit price'] * 0.05, 1) ==
round(data['Tax'], 1)

    # Step 3: Check if all rows pass both Condition_1 and Condition_2
    # Success indicates data integrity; failure suggests potential issues.
    failed_condition_1 = data[~data['Condition_1']]
    failed_condition_2 = data[~data['Condition_2']]

    if failed_condition_1.shape[0] > 0 or failed_condition_2.shape[0] > 0:
        print(f"Data integrity check failed! {failed_condition_1.shape[0]} rows
failed Condition_1, "
```

```
                f"{failed_condition_2.shape[0]} rows failed Condition_2.")
    else:
        print("Data integrity check was successful! All rows pass the integrity
conditions.")

    return data

processed_data = load_and_check()
```

```
Data loaded successfully.
Data integrity check was successful! All rows pass the integrity conditions.
```